

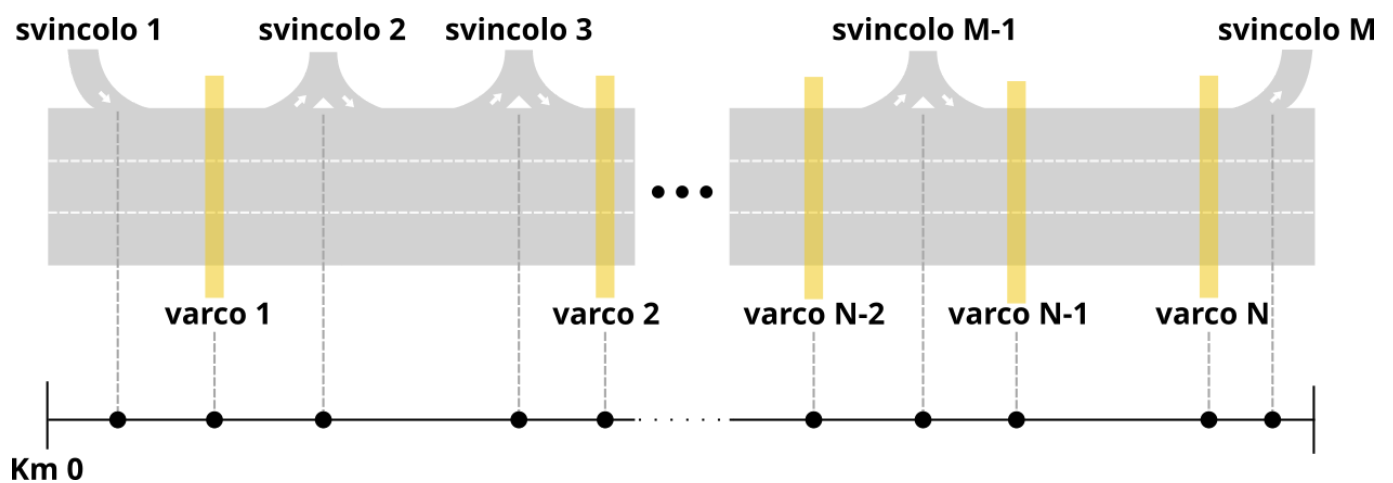
Corso di Laboratorio di Programmazione

Progetto finale – Tutor autostrade

Questo progetto richiede di sviluppare il software per gestire un sistema tutor di controllo dei veicoli che transitano su un'autostrada. L'autostrada è dotata di due o più varchi che registrano il passaggio dei veicoli e di svincoli in corrispondenza dei quali i veicoli possono entrare o uscire. A partire da questi dati il sistema calcola la velocità media e tiene traccia dei veicoli che superano i limiti di velocità.

Per testare il sistema tutor è necessario inoltre sviluppare un simulatore che genera il moto dei veicoli e i dati relativi ai passaggi in corrispondenza dei varchi. Per ogni veicolo il simulatore determina in maniera casuale l'ingresso in autostrada, l'uscita e il profilo di velocità.

Simulatore e sistema tutor sono due eseguibili diversi.



1. Struttura dell'autostrada

L'autostrada è descritta a partire da un'origine (convenzionalmente posta al km 0) rispetto alla quale sono disposti i varchi del sistema tutor e gli svincoli di ingresso/uscita, per ciascuno dei quali è nota la distanza dall'origine. Tutti i veicoli hanno un unico senso di marcia. L'autostrada è descritta da un file di testo denominato `Highway.txt` da inserire nella directory `Data`, che si trova nella root del progetto. Il file `Highway.txt` contenente righe codificate nel modo seguente:

```
<distanza in km> <V|S per Varco o Svincolo>
```

Questo file è pensato per essere fornito al sistema come input, perciò si suppone che sia scritto a mano.

La descrizione deve soddisfare i seguenti vincoli:

- ciascun varco e ciascuno svincolo è identificato da un numero univoco che è assegnato automaticamente dal sistema in ordine crescente di distanza dall'origine - le numerazioni di varchi e svincoli sono indipendenti (cioè saranno presenti il varco 1 e lo svincolo 1);
- devono essere presenti almeno due varchi;

- deve essere presente almeno uno svincolo precedente al primo varco e almeno uno svincolo successivo all'ultimo varco;
- la distanza minima tra uno svincolo e un varco è di 1 km;
- varchi e svincoli possono essere elencati in qualsiasi ordine (non sono necessariamente ordinati nel file).

Descrizioni che non soddisfano i vincoli elencati devono essere rifiutati dal sistema, che quindi può uscire con un errore in caso di file non conforme.

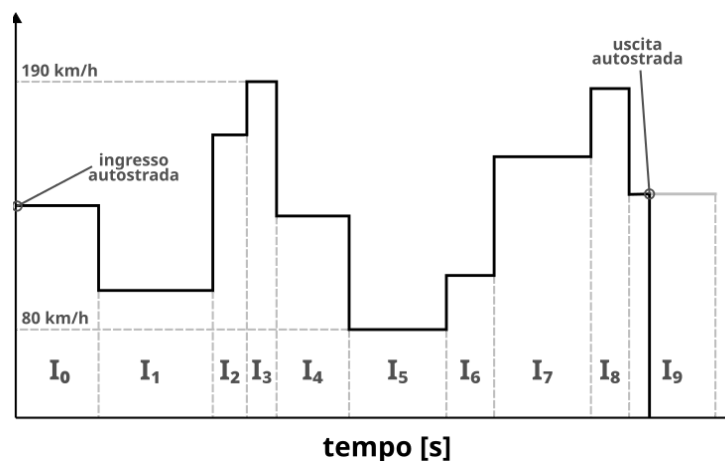
2. Simulatore

2.1 Generazione dei percorsi dei veicoli

Il simulatore genera un determinato numero di veicoli scegliendo casualmente:

- la targa nel formato [due lettere - tre cifre - due lettere]
- lo svincolo di ingresso;
- lo svincolo di uscita;
- data e orario di ingresso;
- il profilo di velocità.

Il profilo di velocità è determinato considerando intervalli consecutivi percorsi a velocità costante; un generico intervallo i -esimo è percorso a velocità v_i per un tempo t_i . I valori di v_i e t_i sono scelti casualmente utilizzando una funzione di generazione di numeri casuali per estrarre valori interi in un range definito da due variabili costanti; si richiede di inizializzare tali costanti considerando il range [80-190] km/h per le velocità e [5-15] minuti per i tempi. Il simulatore deve generare un numero di



intervalli sufficiente per coprire la distanza tra ingresso e uscita. Per semplicità si consideri istantanea la variazione di velocità tra un intervallo e il successivo. Come mostrato in figura, all'ingresso in autostrada il veicolo parte viaggiando alla velocità definita al primo intervallo, mentre all'uscita la velocità passa istantaneamente dal valore dell'ultimo intervallo a 0. L'ultimo intervallo può non essere percorso per tutto il relativo tempo t_i se questo porta il veicolo oltre il casello di uscita.

Il riferimento temporale del simulatore è gestito tramite valori `double` che rappresentano il numero di secondi dall'istante 0 corrispondente al momento in cui il simulatore viene avviato. Gli istanti di partenza dei veicoli sono generati considerando un tempo tra la partenza di un veicolo e quella del successivo scelto casualmente tramite una funzione di generazione di numeri casuali che estrae valori in un range definito da due variabili `double` costanti; si richiede di inizializzare tali costanti considerando il range [0.5-10.0] secondi.

Il numero complessivo di veicoli da simulare è indicato tramite una variabile costante inizializzata al valore 10000. Il sistema potrebbe essere testato anche con valori superiori.

I percorsi generati dovranno essere salvati su un file di testo denominato `Runs.txt`, da inserire nella directory `Data`, le cui righe hanno il seguente formato:

```
<targa> <#   svincolo   ingresso> <#   svincolo   uscita> <istante di partenza> <v0 t0>, <v1 t1>, ..., <vN-1 tN-1>
```

2.2 Generazione del passaggio attraverso i varchi

A partire dai percorsi, il simulatore calcola, per ogni veicolo, il momento in cui questo passa attraverso tutti i varchi che incontra nel proprio tragitto. Questi dati sono salvati in un file denominato `Passages.txt`, di nuovo da inserire nella directory `Data`, che ha il seguente formato:

```
<# varco> <targa> <istante di passaggio>
```

Questo file rappresenta l'input per il sistema tutor.

3. Tutor autostradale

Il sistema di controllo dei transiti dei veicoli analizza i dati prodotti al punto precedente (sezione 2.2) e calcola la velocità media impiegata da ciascun veicolo per percorrere ciascuna tratta compresa tra due varchi consecutivi, confrontandola con il limite di velocità di 130 km/h.

Il sistema parte all'istante 0 e rimane in attesa di input dall'utente. L'utente può inviare un comando che indica che il sistema deve portarsi a un nuovo istante temporale (successivo a quello corrente), analizzando tutti i transiti che avvengono nel tempo intercorso. L'utente può inviare i seguenti comandi:

```
set_time <istante>           // spostamento a un nuovo istante temporale
                               // (errore se è anteriore a quello corrente)
                               // e analizza i transiti tra l'istante
                               // corrente e quello indicato

reset                         // resetta completamente il sistema

stats                         // stampa le statistiche (vedi sotto)
```

L'argomento del comando `set_time` identifica di quanto tempo il simulatore deve avanzare rispetto all'istante corrente. Può essere espresso come un numero puro, nel qual caso si considera misurato in secondi, oppure con un numero seguito da 'm' senza spazi, nel qual caso si considera misurato in minuti. Ad esempio:

```
set_time 20                   // avanzamento di 20 secondi
set_time 20m                  // avanzamento di 20 minuti
```

In risposta a ogni `set_time` il sistema stampa su standard output l'elenco dei veicoli che hanno superato il limite di velocità e pertanto devono essere sanzionati. Ciascuna segnalazione deve essere riportata con i seguenti dati:

- targa;
- tratta (varco di inizio e fine misurazione) su cui è avvenuta la violazione;
- velocità media;
- istanti di passaggio per i due varchi.

Il comando `reset` riporta il sistema allo stato iniziale.

In risposta al comando `stats` il sistema fornisce in output:

- per ogni varco, il numero complessivo di veicoli transitati e il numero medio di veicoli al minuto;
- la velocità media dei veicoli;
- il numero di veicoli sanzionati.

Si evidenzia che, dato l'elevato numero di veicoli in transito in autostrada, è importante effettuare una scelta opportuna per la struttura dati utilizzata per gestire le informazioni.

4. Note per lo sviluppo

Il sistema sviluppato deve essere in grado di gestire una qualunque autostrada descritta secondo le linee guida della sezione 1, pertanto si suggerisce di effettuare test con varie configurazioni di `Highway.txt`.

Il progetto sviluppato **deve** essere gestito tramite CMake e compilabile sulla macchina virtuale (VLab/Taliercio.2020). Non è necessario sviluppare tutto sulla macchina virtuale, ma è importante fare verifiche periodiche per essere sicuri di non aver utilizzato codice fuori standard.

Il progetto deve essere suddiviso in più file sorgente. **Ogni file deve essere scritto da un solo membro del gruppo.** È tuttavia possibile controllare che il codice dei propri compagni di gruppo funzioni correttamente, poiché un errore in un file che inficia il funzionamento del progetto potrebbe causare una penalizzazione della valutazione dell'intero gruppo. **Il nome dell'autore deve essere indicato in un commento a inizio file.** Ovviamente, è necessario e positivo che si discuta all'interno di ciascun gruppo su come realizzare il codice, ma ogni membro è responsabile della gestione del codice che deve scrivere.

Il codice deve essere adeguatamente commentato. **Il software deve essere basato unicamente sulla libreria standard del C++.**

5. Utilizzo AI, plagio, consegna

Il codice verrà controllato per verificare eventuali plaghi tra gruppi appartenenti allo stesso canale e in canali diversi. Verrà effettuato anche un controllo rispetto a codice disponibile online.

Non è consentito inserire nei file codice che non è stato scritto dall'autore dichiarato; inserire codice scritto da qualsiasi AI è vietato.

In caso di dubbi, gli autori potrebbero essere chiamati a un confronto orale con la commissione d'esame.

Il progetto deve essere consegnato su moodle (consegna di gruppo come per la prova intermedia).

Il progetto dovrà essere compresso in un archivio che include una sola directory contenente:

- il codice sorgente (eventualmente organizzato in sottodirectory);
- il file `CMakeLists.txt` necessario alla compilazione (uno solo e posto nella directory principale del progetto);
- la directory `Data` contenente i file `Highway.txt`;
- un file di log di esempio di output della generazione dei percorsi (sezione 2.1, file `Runs.txt`) e un file di esempio di generazione dei passaggi attraverso i varchi (sezione 2.2, file `Passages.txt`) - **tali file possono essere molto grandi: è possibile tagliarli per permettere il caricamento su moodle**;
- un file `README.txt` in cui sono riportate eventuali note che si desidera comunicare in fase di correzione. Questo file non è la documentazione, che deve essere inserita sotto forma di commenti nel codice, ma un elenco di note aggiuntive per chi corregge, per esempio problemi riscontrati e non risolti.

L'archivio non deve contenere gli eseguibili, perché il sorgente sarà compilato in fase di correzione. Il sistema CMake deve compilare con le opzioni di ottimizzazione attivate (-O2).

Dopo la consegna su moodle, **è necessario verificare ciò che è stato consegnato** con i passi seguenti:

- Scaricare il progetto da moodle in una directory diversa da quella usata per sviluppare;
- Lanciare CMake;
- Compilare il codice e verificarne la corretta esecuzione.

Si suggerisce di consegnare anche compiti non completi. È tuttavia necessario che il software consegnato sia **compilabile** ed **eseguibile**.

La consegna in ritardo degli elaborati sarà **fortemente penalizzata** e considerata solo in un brevissimo lasso temporale dopo la scadenza (pochi minuti). Consegne effettuate successivamente saranno ignorate.

Saranno valutati:

- Chiarezza e correttezza del codice;
- Corretta gestione della memoria e delle strutture dati;
- Efficienza del codice e della soluzione trovata;
- Utilizzo di strumenti appropriati.

Alcuni esempi (non esaustivi) delle voci nella griglia di valutazione del progetto:

- Il progetto non compila
- Il progetto non esegue (dà sempre segmentation fault)
- Il progetto produce segmentation fault occasionali
- Il progetto ha memory leak
- CMakeLists sbagliato o inesistente
- Erroneo (assente o eccessivo) uso dell'ereditarietà
- Erroneo uso della standard library per gestire elementi multipli
- Mancato controllo bound array / accesso a memoria non consentito
- Mancato controllo argomenti da riga di comando

- Utilizzo allocazione dinamica in maniera non opportuna
- Il programma non funziona correttamente
- Bad code style lieve/grave
- Mancanza log file richiesto dal progetto